

The Economic Order Quantity (EOQ)

The EOQ answers one specific question: if demand is steady and predictable, how many units should you order each time you place an order?

Notice that's different from Week 1's question. Week 1 asked 'how much stock should I hold per day?' EOQ asks 'when I do order, how big should that order be?'

The two are connected -- bigger orders mean you order less frequently but hold more inventory on average.

The Setup

The EOQ model makes a few simplifying assumptions. These aren't perfectly realistic, but they give us a clean mathematical problem to solve, and the answer is surprisingly useful even in messy real-world situations.

Assumption	What it means
Demand is constant	Customers buy the same number of units every day
Demand rate is known	We know exactly how many units per year (or per day) will be sold
Lead time is zero	When you place an order, it arrives immediately (we'll relax this in Week 3)
No shortages allowed	You always have enough stock i.e. you never run out
Ordering cost is fixed	Every order costs the same fixed amount K , regardless of order size

The Three Parameters

EOQ has exactly three inputs.

Symbol	Name	What it is	Typical example
D	Annual demand	Total units sold per year	1,300 units/year
K	Fixed ordering cost	Cost to place one order — paperwork, delivery setup, supplier minimum fees	₹500 per order placed
h	Holding cost	Cost to store one unit for one year — warehouse rent, insurance, capital tied up	₹18 per unit per year

Deriving the Formula

The key insight is that two costs move in opposite directions as you change order size Q:

1

Annual ordering cost goes DOWN as Q goes UP

If you order Q units each time, you place D/Q orders per year. Each order costs K. So annual ordering cost = $K \times (D/Q)$. Bigger Q → fewer orders → lower ordering cost.

2

Annual holding cost goes UP as Q goes UP

If you order Q units at once, your inventory cycles between Q (just after an order) and 0 (just before). On average you hold $Q/2$ units. Each unit costs h per year. So annual holding cost = $h \times (Q/2)$. Bigger Q → more average inventory → higher holding cost.

3

Total cost has a minimum — that's Q^*

Total cost = $K(D/Q) + h(Q/2)$. As Q increases, the first term falls and the second rises. There's a crossover point where their sum is minimised. Setting the derivative to zero (or just drawing the curves) gives: $Q^* = \sqrt{(2KD/h)}$. That's the EOQ formula.

The formula:

$$Q^* = \sqrt{(2 \times K \times D / h)}$$

$$\text{Minimum total annual cost} = \sqrt{(2 \times K \times D \times h)}$$

An important property of EOQ

EOQ is surprisingly robust. If you get D wrong by 20%, Q^* only changes by about 10%. This is because Q^* depends on the square root of D - errors get 'dampened'. In practice this means even rough estimates of demand give a pretty good order quantity. This is why EOQ, despite its simplifying assumptions, is still widely used in industry.

What to Watch

Video	Watch it when
youtube.com/watch?v=AYpjPWmlyHM	derives EOQ step by step Pause and verify each step yourself.
youtube.com/watch?v=luwrCrUmvwc	After the first video.

Part A : EOQ in Plain Python

We start without PuLP. The goal here is to implement the EOQ formula directly and build intuition through the plots and sensitivity table. This is the same approach as Week 1 but now the math is telling you exactly what to do instead of you guessing.

The Complete Code

```
# WEEK 2 PART A - EOQ in plain Python

import numpy as np
import matplotlib.pyplot as plt

# Our three parameters
D = 1300          # annual demand: we sell 1300 units per year
K = 8             # fixed cost per order: each order costs $8 to place
h = 0.225         # holding cost: storing one unit for one year costs $0.225

# The EOQ formula
Q_star = (2 * K * D / h) ** 0.5
min_cost = (2 * K * D * h) ** 0.5

print('=== EOQ Results ===')
print(f'Optimal order quantity Q* = {Q_star:.1f} units')
print(f'Minimum annual total cost = ${min_cost:.2f}')
print(f'Number of orders per year = {D / Q_star:.1f}')
print(f'Days between orders = {365 / (D / Q_star):.0f} days')

# At Q* the two costs should be exactly equal
annual_ordering = K * (D / Q_star)
annual_holding = h * (Q_star / 2)
print(f'\nAt Q*:')
print(f' Annual ordering cost = ${annual_ordering:.2f}')
print(f' Annual holding cost = ${annual_holding:.2f}')
print(f' These are equal. That is exactly the EOQ condition.')

# Plot the three cost curves
Q_range = np.linspace(50, 800, 400)
cost_ordering = K * (D / Q_range)
cost_holding = h * (Q_range / 2)
cost_total = cost_ordering + cost_holding

plt.figure(figsize=(9, 5))
plt.plot(Q_range, cost_ordering, 'b--', label='Ordering cost K x (D/Q)',
         linewidth=2)
plt.plot(Q_range, cost_holding, 'r--', label='Holding cost h x (Q/2)',
         linewidth=2)
plt.plot(Q_range, cost_total, 'k-', label='Total cost',
         linewidth=2.5)
plt.axvline(Q_star, color='green', linestyle=':', linewidth=2,
            label=f'Q* = {Q_star:.0f} (minimum total cost)')
plt.xlabel('Order Quantity Q (units)')
plt.ylabel('Annual Cost ($)')
plt.title('EOQ: Ordering Cost vs Holding Cost Trade-off')
plt.legend()
plt.grid(True, alpha=0.3)
plt.ylim(0, 300)
plt.show()
```

```
# Sensitivity: what if our demand estimate is wrong?
print('\n=== What if we estimate D wrongly? ===')
print(f'{'True D':>10} {'Q* we should use':>18} {'Q* we actually use':>20}
{'Extra cost':>12}')
for true_D in [900, 1000, 1100, 1200, 1300, 1400, 1500, 1600, 1700]:
    true_Q = (2 * K * true_D / h) ** 0.5
    true_TC = K * (true_D / true_Q) + h * (true_Q / 2)
    our_TC = K * (true_D / Q_star) + h * (Q_star / 2)
    extra = our_TC - true_TC
    print(f'{'true_D':>10} {'true_Q':>18.1f} {'Q_star':>20.1f} ${extra:>10.2f}')
print('\nNotice how small the extra cost is even when D is quite wrong.')
print('This is why EOQ is called robust.')
```

4. What is a Linear Program?

A linear program is a mathematical problem with three parts:

1

Decision variables

The things you are choosing. In inventory problems these are usually quantities: how many units to order, how much stock to hold. You do not know their values at the start. The solver finds them.

2

Objective function

What you want to minimise or maximise. In inventory problems this is almost always total cost. The objective must be a linear expression: something like $3x$ plus $5y$, not x squared or x times y .

3

Constraints

Rules the solution must follow. A warehouse has limited space. You cannot hold negative inventory. Demand must be met. Each constraint is an inequality or equality that the decision variables must satisfy.

The key word is linear

Every term in the objective and constraints must be linear, meaning no squares, no products of two variables, no square roots. This sounds limiting but it is actually a strength. Linear programs can be solved exactly and extremely quickly, even with thousands of variables. The EOQ formula involved a square root, which is why it was nonlinear and we could solve it with calculus. When we move to multi-period, multi-node models in Week 3, the problems are naturally linear and PuLP handles them perfectly.

Part B : Your First LP in PuLP

Now we will solve a simple inventory-style LP in PuLP. This problem is not EOQ. It is a deliberately simple made-up problem whose only purpose is to teach you PuLP syntax on familiar ground before Week 3 introduces new inventory concepts at the same time.

The Problem

A small shop has 100 units of shelf space. It stocks two products, A and B. Each unit of A earns a profit of 40 rupees. Each unit of B earns a profit of 30 rupees. The shopkeeper has a fixed supply budget that means total units ordered cannot exceed 80. How many of each product should the shopkeeper stock to maximise profit?

Written as an LP:

Part of the LP	How it looks
Decision variables	x_A = units of Product A to stock, x_B = units of Product B to stock
Objective	Maximise $40 x_A + 30 x_B$ (total profit)
Constraint 1	$x_A + x_B \leq 100$ (shelf space limit)
Constraint 2	$x_A + x_B \leq 80$ (budget limit)
Constraint 3	$x_A \geq 0, x_B \geq 0$ (cannot stock negative units)

Try to guess the answer before running the code. Should the shopkeeper stock only A, only B, or a mix? Why?

The Complete PuLP Code

```
# WEEK 2 PART B - First LP in PuLP
# If PuLP is not installed yet, run this once in a terminal:
# pip install pulp

import pulp

# Step 1: Create an LP problem and say we want to maximise
prob = pulp.LpProblem('ShopLP', pulp.LpMaximize)

# Step 2: Define decision variables
x_A = pulp.LpVariable('x_A', lowBound=0) # units of Product A
x_B = pulp.LpVariable('x_B', lowBound=0) # units of Product B

# Step 3: Add the objective function
prob += 40*x_A + 30*x_B

# Step 4: Add the constraints
prob += x_A + x_B <= 100, 'shelf_space'
prob += x_A + x_B <= 80, 'budget'

# Step 5: Solve
prob.solve(pulp.PULP_CBC_CMD(msg=0))
```

```
# Step 6: Read the results
print('=== Solution ===')
print(f'Status : {pulp.LpStatus[prob.status]}')
print(f'x_A : {x_A.varValue:.0f} units of Product A')
print(f'x_B : {x_B.varValue:.0f} units of Product B')
print(f'Profit : Rs {pulp.value(prob.objective):.0f}')
print(f'\nShelf space used : {x_A.varValue + x_B.varValue:.0f} out of 100')
print(f'Budget used : {x_A.varValue + x_B.varValue:.0f} out of 80')
```

How These Two Things Connect

You might be wondering what a shopkeeper profit problem has to do with EOQ. The connection is this: both are optimisation problems. In EOQ you are minimising total cost over one variable Q . In the LP you are maximising profit over two variables x_A and x_B . PuLP can solve both.

Think of this week as two separate things happening in parallel. EOQ teaches you the inventory intuition: ordering cost and holding cost trade off, there is an optimal order size, and the formula finds it. PuLP teaches you the tool: define variables, write objective, add constraints, solve.

Experiments

Experiment 1 - EOQ: What if ordering is very cheap?

Change K from 8 to 1. Re-run the EOQ code. How does Q^* change? Are you ordering more or less frequently? Does this make sense? Now try K equal to 80 (very expensive to order). What happens?

Experiment 2 - EOQ: What if holding is very expensive?

Change h from 0.225 to 3.0. Think about what this represents physically. Maybe you are storing refrigerated goods that cost a lot to keep cold. What happens to Q^* ? Are you ordering bigger or smaller quantities now?

Experiment 3 - PuLP: Add a third product

Add a Product C that earns 35 rupees profit per unit and uses the same shelf space. Add it to the LP. What does the solver do? Does it stock any C at all? Why or why not? What would you need to change to make the solver want to stock some C?

Experiment 4 - PuLP: Change maximise to minimise

Change `LpMaximize` to `LpMinimize` in the problem definition. Re-run without changing anything else. What answer do you get? Does it make sense? This experiment makes sure you understand what the objective direction actually controls.

Deliverable

The above experiments and your understanding of PuLP in the report.

